OCEAN EXPLORER KATA

A Test-Driven Development Engineering Report

Java | TDD | Clean Code | OOP | JUnit 5 | Maven

SUBMISSION BY:	**Pallavi Metri**
LANGUAGE & FRAMEWORK	**Java 17 · JUnit 5 · Maven**
METHODOLOGY	**Test-Driven Development (TDD)**
GitHub	https://github.com/metripallavi/Ocean-

1. Abstract / Executive Summary

This report documents the design, implementation, and testing of the Ocean Explorer Kata — a Java-based software engineering exercise developed using Test-Driven Development (TDD). The system models an autonomous probe navigating a bounded ocean grid, capable of moving forward and backward, turning in cardinal directions, avoiding obstacles, and processing command sequences programmatically.

The project was built incrementally over 25 discrete steps, each driven by a failing test before any production code was written. This document presents the engineering rationale behind key design decisions, the evolution of the class architecture, and the comprehensive testing strategy employed — reflecting professional engineering standards expected in industry environments.

The result is a clean, maintainable, and fully tested Java API adhering to SOLID principles, with clear separation of concerns across four principal classes: **Probe**, **OceanGrid**, **Direction**, and **Coordinate**.

Metric	Value
Development Steps	25 incremental TDD cycles
Test Coverage	All behaviours, edge cases & validation
Primary Language	Java 17
Build Tool	Apache Maven
Test Framework	JUnit 5
Design Pattern	OOP + SOLID + Clean Code

2. Problem Statement

The Ocean Explorer Kata requires the engineering of a simulation API for an autonomous underwater probe. The probe must operate within a defined rectangular ocean grid and respond to a sequence of directional commands. The problem encompasses several interconnected challenges:

Positional Awareness	The probe must maintain an accurate internal representation of its current (x, y) coordinates and cardinal direction (NORTH, SOUTH, EAST, WEST) at all times.
Directional Movement	The probe must move forward or backward relative to the direction it is currently facing, correctly updating coordinates for all four orientations.
Rotation	The probe must turn left (anticlockwise) or right (clockwise) and correctly update its facing direction through all four cardinal states.
Grid Boundary Enforcement	Movement must be constrained within the defined grid dimensions. Attempts to move beyond the boundary must be silently ignored, preserving the probe's position.
Obstacle Avoidance	The grid may contain obstacles at specified coordinates. The probe must not enter a cell occupied by an obstacle.
Command Execution	The probe must accept and parse a string of sequential commands (F, B, L, R) and execute them in order, with robust handling of invalid or null input.
Input Validation	All inputs — probe construction parameters, grid dimensions, obstacle coordinates, and command strings — must be validated with meaningful exceptions on failure.

3. Approach & Design Philosophy

3.1 Test-Driven Development

The entire project was built using **Test-Driven Development (TDD),** a discipline that inverts the traditional coding workflow. Rather than writing implementation code first and testing afterwards, TDD demands that every feature begins with a *failing test* that defines the expected behaviour precisely. Only then is the minimum implementation code written to satisfy that test.

■ RED	■ GREEN	■ REFACTOR
Write a failing test that defines the desired behavior.	Write the minimum code to make the test pass.	Clean up the code without breaking any tests.

This cycle was applied **25 times** throughout the project — one iteration per development step. The discipline of this approach ensures that every line of production code is justified by a test requirement, and that the design emerges naturally from the specification rather than being imposed prematurely.

3.2 Incremental Development

The system was constructed in small, verifiable increments. Each step introduced a single, well-defined behavior and was fully tested before the next step commenced. This approach has several advantages:

- Failures are isolated — a broken test points directly to the most recently added behavior.
- The codebase remains compliable and runnable at every stage of development.
- Complexity is introduced gradually, preventing over-engineering at early stages.
- Each step produces a working, demonstrable deliverable.

3.3 Design Decisions

Delayed Abstraction: Supporting classes (OceanGrid, Coordinate) were not introduced until tests demanded them. This avoids speculative abstraction and keeps early code simple and readable.

Behaviour in the Right Class: Movement logic belongs in Probe; boundary and obstacle logic belongs in OceanGrid. This respects the Single Responsibility Principle.

Immutability Where Appropriate: Initial coordinates are validated in the constructor. The direction field is mutable (as turning changes it) but validated on construction.

Defensive Validation: Guard clauses at all entry points (constructor, execute (), addObstacle()) prevent invalid state from ever entering the system.

4. System Design

4.1 Class Architecture Overview

The final system comprises four classes, each with a clearly defined responsibility. The design follows the **Single Responsibility Principle** and **Separation of Concerns** to ensure each class can be modified, extended, or tested independently.

Class	Responsibility	Key Methods / Fields
Probe	Central entity. Encapsulates probe state (position) direction and behavior.	moveForward(), moveBackward(), execute (String commands), getX(), getY(), getDirection()
OceanGrid	Represents the bounded ocean grid. Manage dimensions and obstacle positions.	addObstacle(x, y), hasObstacle(x, y), getWidth(), getHeight ()
Direction	Enum representing the four cardinal directions.	NORTH, SOUTH, EAST, WEST (with utility methods for turning if applicable)
Coordinate	Value object represents an (x, y) position.	equals(), hashCode()

4.2 Class Relationships

Probe OceanGrid**:** Probe holds a reference to OceanGrid and delegates all boundary and obstacle checking to it. Probe never computes boundaries directly — this is OceanGrid's sole domain.

OceanGrid Coordinate: OceanGrid uses a Set<Coordinate> to store obstacle positions. The Coordinate value object provides correct equality semantics via overridden equals () and hashCode(), enabling O(1) lookup.

Probe Direction: Probe stores its current facing as a Direction enum value, providing type safety and preventing illegal direction values from ever existing in the system.

4.3 Movement Coordinate System

Direction	moveForward()	moveBackward()
NORTH	y + 1	y □ 1
SOUTH	y □ 1	y + 1
EAST	x + 1	x □ 1

WEST	x □ 1	x + 1

5. Implementation Walkthrough

The following section details each development step, presenting the test written, the code produced, and the design reasoning applied. This is the core TDD narrative of the project.

Step 0 — Project Setup & Environment Validation

Before writing a single line of business logic, the Maven project structure was established and validated. The standard Maven layout was created (src/main/java/com/oceanexplorer and src/test/java/com/oceanexplorer), with pom.xml configured for Java 17 and JUnit 5.

Running mvn test at this stage — with no source files — confirmed the build toolchain was fully operational. This is an often-overlooked but essential step: it establishes a clean baseline and proves the test runner is wired correctly before any complexity is introduced.

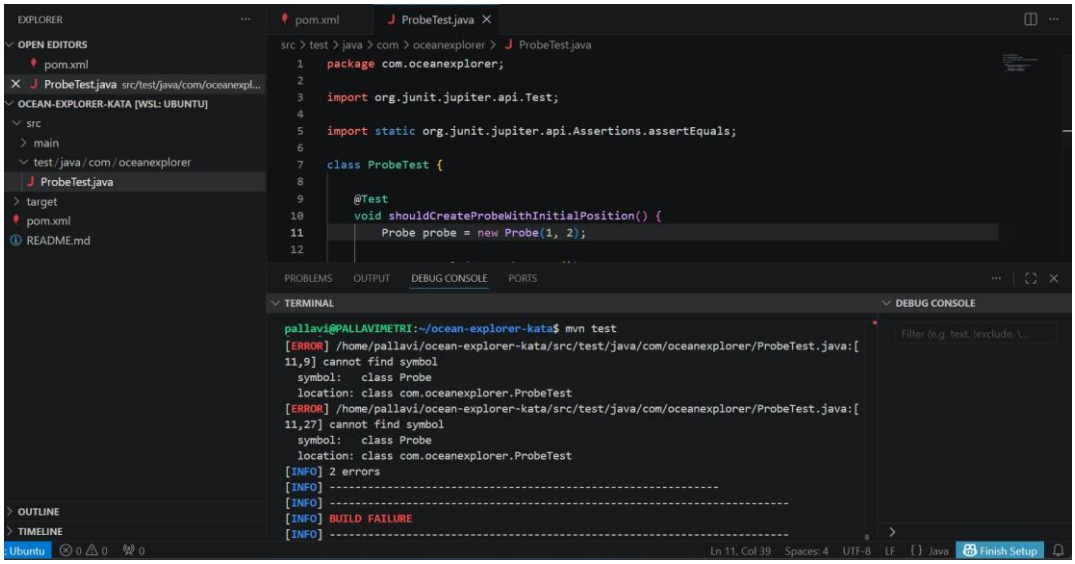


Figure 1 — Initial build failure (no source): confirming the test runner is active

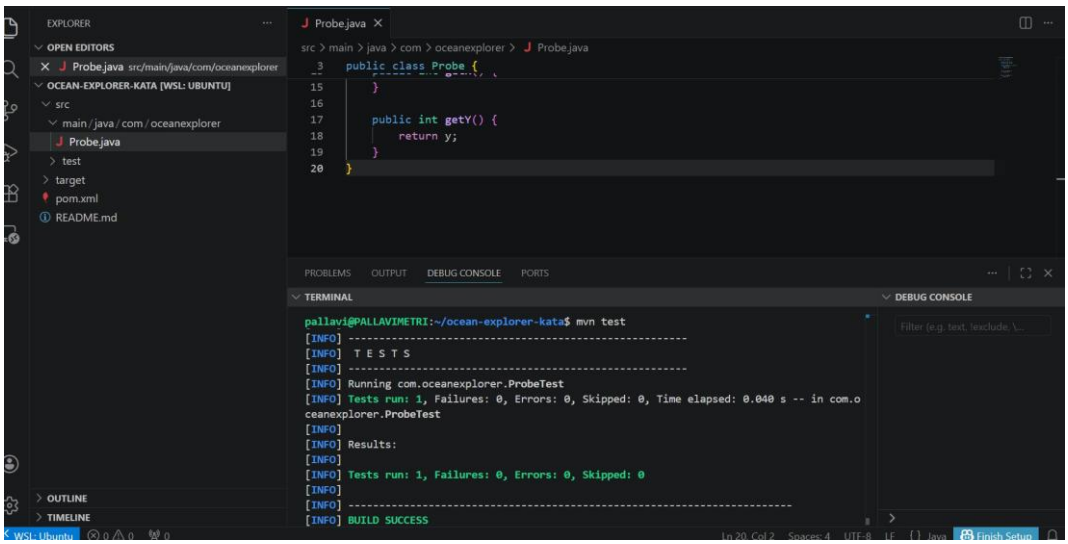


Figure 2 — Build SUCCESS after creating the package structure

Step 1 — Probe with Position

The first test defined the most fundamental behaviour: a probe must know where it is. Rather than designing a fully featured class upfront, TDD demanded only what the test required.

Test written:

```
Probe probe = new Probe(1, 2); assertEquals(1, probe.getX()); assertEquals(2,
probe.getY());
```

The implementation introduced the Probe class with two final fields (x, y) and corresponding getters. The fields were declared final because at this stage position was immutable — movement had not yet been specified. This reflects the TDD maxim: *only build what the test requires*.

Step 2 — Direction

The next test required the probe to know which way it was facing. A Direction enum was introduced with values NORTH, SOUTH, EAST, WEST — providing compile-time safety that a plain String or integer could never offer. The Probe constructor was extended to accept a Direction parameter.

> *Using an enum here is a deliberate design choice: it makes invalid directions impossible to represent in the type system, eliminating an entire class of runtime errors before they can occur.*

Step 3 — Turn Left

Test written:

```
probe.turnLeft(); assertEquals(Direction.WEST, probe.getDirection()); // NORTH →
WEST
```

This step introduced the turnLeft() method. The test first failed with a compilation error (*turnLeft() does not exist*) — the expected Red phase. The switch expression was used to map each direction to its anticlockwise neighbour: NORTH□WEST□SOUTH□EAST□NORTH. The direction field was changed from final to mutable, since turning is a state change.

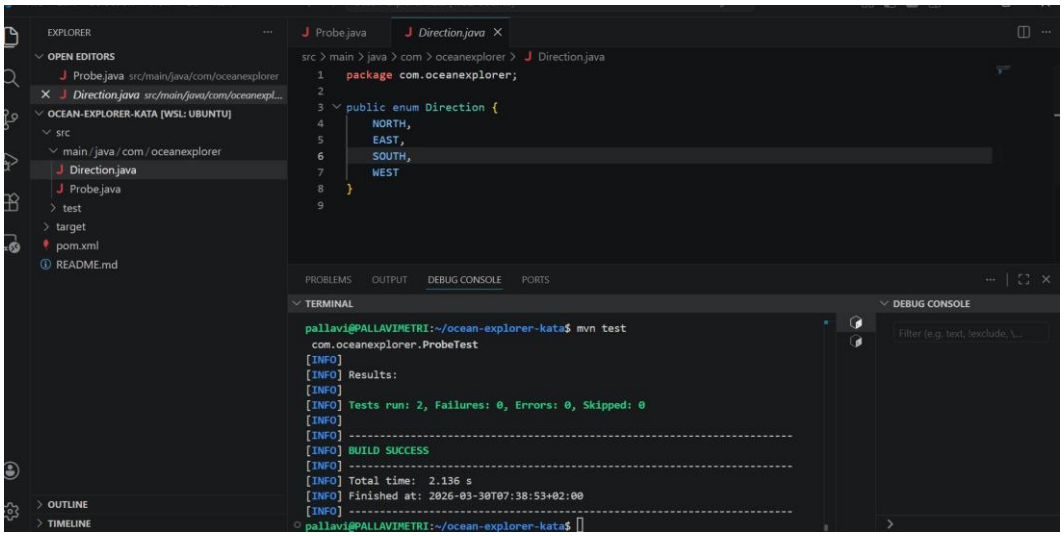


Figure 3 — Build SUCCESS after implementing turnLeft()

Step 4 — Turn Right

By symmetry, turnRight()was added following the same TDD cycle: NORTH, EAST,SOUTH , WEST , NORTH. The step also surfaced a future refactoring candidate — both turn methods contained direction-mapping logic. This duplication was noted and accepted at this point (the system was still small), with a conscious decision to refactor it into the Direction enum in a later step.

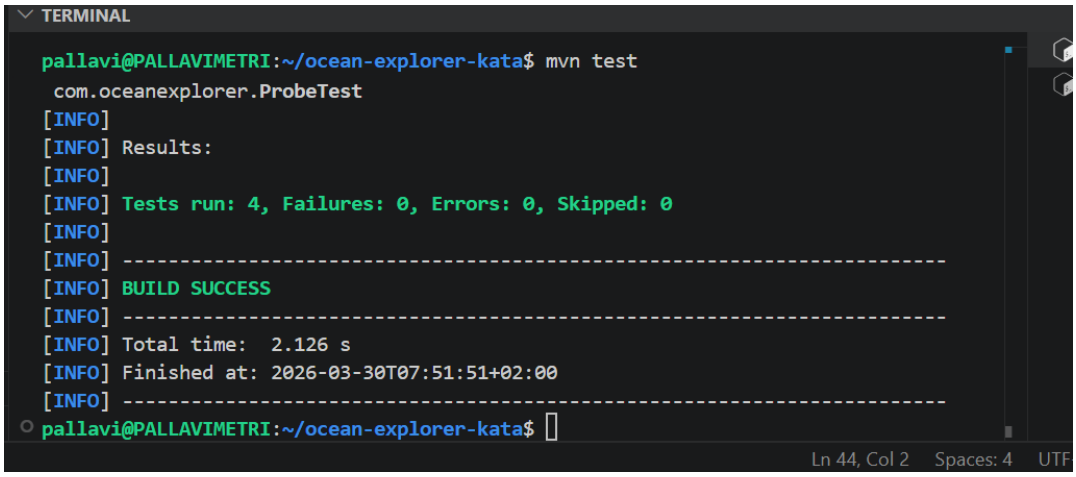


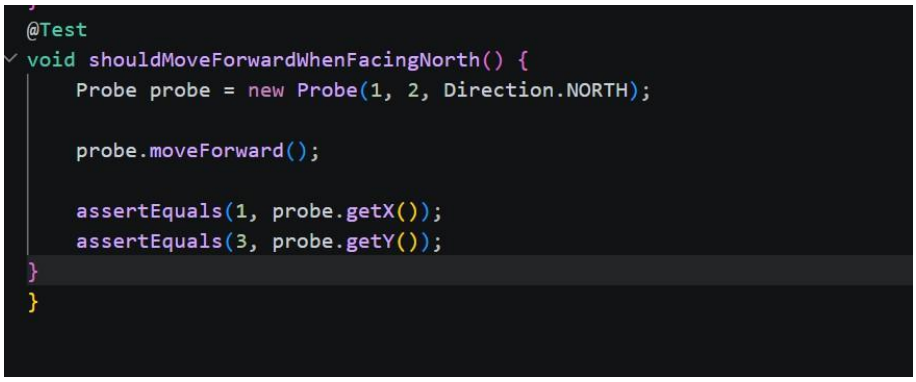
Figure 4 — Build SUCCESS after implementing turnRight()

Step 5 — Move Forward

Forward movement was the first test requiring coordinate mutation. The x and y fields were changed from final to mutable. The moveForward() method used a switch expression to update the appropriate coordinate based on direction.

Test:

```
Probe probe = new Probe(0, 0, Direction.NORTH); probe.moveForward(); assertEquals(1,
probe.getY());
```



```
@Test
void shouldMoveForwardWhenFacingNorth() {
    Probe probe = new Probe(1, 2, Direction.NORTH);

    probe.moveForward();

    assertEquals(1, probe.getX());
    assertEquals(3, probe.getY());
}
}
```

Figure 5 — Test for moveForward() (facing NORTH increases Y)

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    PORTS
TERMINAL
pallavi@PALLAVIMETRI:~/ocean-explorer-kata$ mvn test
 com.oceanexplorer.ProbeTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.991 s
[INFO] Finished at: 2026-03-30T07:56:18+02:00
[INFO] ------------------------------------------------------------------------
pallavi@PALLAVIMETRI:~/ocean-explorer-kata$
Ln 39, Col 2    Spaces: 4    UTF-8    LF
```

Figure 6 — Build SUCCESS after implementing moveForward()

Step 6 — Move Backward

Backward movement is the logical inverse of forward movement. Rather than duplicating the coordinate logic, moveBackward() applied the opposite delta for each direction. A dedicated test verified that a NORTH-facing probe facing backward decreases its Y coordinate.

```
public class Probe {
    }
}
public void moveBackward() {
    switch (direction) {
        case NORTH -> y--;
        case EAST -> x--;
        case SOUTH -> y++;
        case WEST -> x++;
    }
}
```

Figure 7 — moveBackward() implementation snippet

Step 7 — Grid Boundary Handling (OceanGrid Introduced)

This step marked the first significant architectural expansion. Rather than adding boundary logic directly into Probe (which would violate the Single Responsibility Principle), a new class — **OceanGrid** — was introduced to own boundary knowledge.

OceanGrid was constructed with a width and height, and exposed an isWithinBounds(x, y) method. Probe was updated to hold a reference to an OceanGrid, and both moveForward() and moveBackward() were modified to compute the target cell and only update position if the grid confirms it is within bounds.

> *Design principle: Probe asks OceanGrid whether a move is valid. It does not perform boundary arithmetic itself. This is a clean separation of concerns.*

```
Probe.java    ProbeTest.java
src > test > java > com > oceanexplorer > ProbeTest.java
49  void shouldMoveBackwardWhenFacingNorth() {
55      assertEquals(1, probe.getY());
56  }
57  @Test
58  void shouldNotMoveOutsideGrid() {
59      OceanGrid grid = new OceanGrid(5, 5);
60      Probe probe = new Probe(0, 0, Direction.SOUTH, grid);
61
62      probe.moveForward();
63
64      assertEquals(0, probe.getX());
65      assertEquals(0, probe.getY());
66  }
PROBLEMS   OUTPUT   DEBUG CONSOLE   PORTS
TERMINAL
pallavi@PALLAVIMETRI:~/ocean-explorer-kata$ mvn test
[INFO] Compiling 2 source files to /home/pallavi/ocean-explorer-kata/target/classes
[INFO] -------------------------------------------------------------
[ERROR] COMPILATION ERROR :
[INFO] -------------------------------------------------------------
[ERROR] /home/pallavi/ocean-explorer-kata/src/main/java/com/oceanexplorer/Probe.java
:[60,2] class, interface, enum, or record expected
[INFO] 1 error
[INFO] -------------------------------------------------------------
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.028 s
Ln 64, Col 35   Spaces: 4   UTF-8
```

Figure 8 — Test and implementation for grid boundary checking

```
pallavi@PALLAVIMETRI:~/ocean-explorer-kata$ mvn test
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.825 s
[INFO] Finished at: 2026-03-30T08:46:05+02:00
[INFO] ------------------------------------------------------------------------
pallavi@PALLAVIMETRI:~/ocean-explorer-kata$
Ln 14, Col 1   Spaces: 4   UTF-8   LF   { } Jav
```

Figure 9 — Build SUCCESS with boundary enforcement

Step 8 — Obstacle Avoidance

OceanGrid was extended to support obstacle storage and lookup. A Set<String> (later refactored to Set<Coordinate>) was added to store blocked cells. Two new methods were introduced: addObstacle(x, y) and hasObstacle(x, y).

Probe's movement methods were updated to check both boundary validity and obstacle presence before committing to a move. The probe silently remains in place if either condition fails — consistent with the specification.

```
Probe.java   ProbeTest.java ×   OceanGrid.java
src > test > java > com > oceanexplorer > ProbeTest.java
 7   class ProbeTest {
79       @Test
80   void shouldNotMoveIntoObstacle() {
81       OceanGrid grid = new OceanGrid(5, 5);
82       grid.addObstacle(0, 1);
83
84       Probe probe = new Probe(0, 0, Direction.NORTH, grid);
85
86       probe.moveForward();
87
88       assertEquals(0, probe.getX());
89       assertEquals(0, probe.getY());
PROBLEMS   OUTPUT   DEBUG CONSOLE   PORTS
TERMINAL
pallavi@PALLAVIMETRI:~/ocean-explorer-kata$ mvn test
java:[82,9] cannot find symbol
[ERROR]   symbol:   method addObstacle(int,int)
[ERROR]   location: variable grid of type com.oceanexplorer.OceanGrid
[ERROR]
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read th
e following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureExcepti
on
pallavi@PALLAVIMETRI:~/ocean-explorer-kata$
```

Figure 10 — Obstacle storage and detection in OceanGrid

Step 9 — Command Execution (execute() Method)

Steps 9 and 10 introduced the execute(String commands) method — the primary external API of the Probe. The method iterates through each character of the command string and dispatches to the appropriate movement or rotation method:

F □ moveForward() | B □ moveBackward() | L □ turnLeft() | R □ turnRight()

Any unrecognised character causes an IllegalArgumentException to be thrown, ensuring the method never silently ignores bad input.

```
pallavi@PALLAVIMETRI:~/ocean-explorer-kata$ mvn test
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.362 s
[INFO] Finished at: 2026-03-30T09:57:22+02:00
[INFO] ------------------------------------------------------------------------
pallavi@PALLAVIMETRI:~/ocean-explorer-kata$
```

Figure 11 — Command Execution Sequence

6. Key Features

Movement — Forward & Backward

- The probe moves one cell at a time in the direction it is facing.
- Both moveForward() and moveBackward() validate the target cell before updating state.
- A shared private move(dx, dy) helper (introduced in Step 25) eliminates duplication.
- Movement is silently ignored if the target is out-of-bounds or contains an obstacle.

Turning — Left & Right

- The probe rotates 90° at a time through the four cardinal directions.
- Switch expressions provide clear, readable state-transition logic.
- Turning does not change the probe's position — only its facing direction.
- All four turn transitions are verified by dedicated tests.

Grid Boundary Enforcement

- OceanGrid defines the valid coordinate space as [0, width) × [0, height).
- isWithinBounds(x, y) is the single authority on position validity.
- Boundary violations do not throw — the probe simply stays in place.
- Grid dimensions are validated in the constructor; zero or negative values are rejected.

Obstacle Avoidance

- Obstacles are registered via addObstacle(x, y), validated to be within grid bounds.
- Obstacle positions are stored in a Set for O(1) lookup efficiency.
- Duplicate obstacle registration is idempotent (Set semantics guarantee this).
- Probe cannot be initialised on a cell that contains an obstacle.

Command Execution

- execute(String commands) processes a full sequence of commands in a single call.
- Command strings are case-normalised to uppercase before processing (Step 19).
- Whitespace within command strings is skipped gracefully (Step 14).
- Null input throws IllegalArgumentException; empty string is a valid no-op.

7. Edge Case Handling

Robust software anticipates and handles all failure modes explicitly. The following edge cases were each addressed with dedicated tests and implementation changes.

Null Command String (Step 11)	Test:	shouldThrowExceptionWhenCommandsAreNull
	Fix:	A guard clause at the top of execute() checks for null and throws IllegalArgumentException immediately.
	Why:	*Prevents NullPointerException propagating from within the method, giving the caller a meaningful error.*
Empty Command String (Step 12)	Test:	shouldDoNothingWhenCommandsAreEmpty
	Fix:	No implementation change required — iterating an empty string produces no iterations.
	Why:	*An explicit test documents and pins this intentional no-op behaviour.*
Invalid Command Character (Step 10)	Test:	shouldThrowExceptionForInvalidCommand
	Fix:	The default branch of the switch in processCommand() throws IllegalArgumentException.
	Why:	*Any character that is not F, B, L, or R is immediately rejected with a clear message.*
Lowercase Commands (Step 13 □ resolved Step 19)	Test:	shouldThrowExceptionForLowercaseCommands □ shouldAcceptLowercaseCommands
	Fix:	Initially lowercase was rejected. Step 19 added toUpperCase() normalisation before processing.
	Why:	*Improved API usability without weakening validation.*
Null Direction in Constructor (Step 16b)	Test:	shouldThrowExceptionWhenDirectionIsNull
	Fix:	Constructor validates direction != null and throws IllegalArgumentException.
	Why:	*Prevents a probe from being created in an uninitialised state.*
Null Grid in Constructor (Step 17)	Test:	shouldThrowExceptionWhenGridIsNull
	Fix:	Constructor validates grid != null and throws IllegalArgumentException.
	Why:	*Prevents NullPointerException at movement time, catching the error at the point of construction.*

Starting Position Out of Bounds (Step 18)	Test:	shouldThrowExceptionWhenStartingOutsideBounds
	Fix:	Constructor calls grid.isWithinBounds(x, y) and throws if invalid.
	Why:	*Guarantees the probe always starts in a reachable, valid position.*
Starting Position on Obstacle (Step 20)	Test:	shouldThrowExceptionWhenStartingOnObstacle
	Fix:	Constructor calls grid.hasObstacle(x, y) after bounds check.
	Why:	*A probe that starts on an obstacle could never move — this prevents nonsensical initial state.*
Invalid Grid Dimensions (Step 21)	Test:	shouldThrowExceptionForZeroWidth / shouldThrowExceptionForNegativeHeight
	Fix:	OceanGrid constructor validates width > 0 and height > 0.
	Why:	*Eliminates the concept of a zero-area or inverted grid from the system entirely.*
Obstacle Outside Grid (Step 22)	Test:	shouldThrowExceptionWhenAddingObstacleOutsideGrid
	Fix:	addObstacle() calls isWithinBounds() before adding to the Set.
	Why:	*Prevents phantom obstacles outside navigable space from corrupting the grid model.*
Duplicate Obstacle (Step 23)	Test:	shouldHandleDuplicateObstaclesGracefully
	Fix:	No code change required — Set semantics handle deduplication automatically.
	Why:	*Demonstrates awareness of data structure properties; no defensive code needed.*
Whitespace in Commands (Step 14)	Test:	shouldIgnoreSpacesInCommands
	Fix:	processCommand() skips characters that are spaces using a continue statement.
	Why:	*Allows human-readable command strings like 'F F R F' to be processed correctly.*

8. Refactoring Decisions

Refactoring is the third phase of the TDD cycle. The following improvements were made after tests were passing, ensuring no regression was introduced.

Step 16a — Extracting processCommand()

The execute() method was becoming a two-concern method: it both iterated commands and processed each one. A private processCommand(char command) method was extracted, delegating single-character dispatch. This made execute() a clean orchestration method and processCommand() a focused handler.

Step 24 — Coordinate Value Object

Obstacle storage originally used String keys (e.g., '2,3') — a fragile approach that relies on string formatting conventions. This was replaced with a proper Coordinate class implementing equals() and hashCode() correctly. The Set is now type-safe and semantically clear.

Step 25 — Shared move(dx, dy) Helper

moveForward() and moveBackward() both contained similar boundary-check and position-update logic. A private move(int dx, int dy) helper was introduced, encapsulating the check-and-update pattern. Both methods now delegate to this helper with the appropriate deltas, eliminating duplication.

direction □ mutable (Step 3)

The direction field was initially declared final. When turning was introduced, it was changed to mutable. This reflects a deliberate design evolution — not all fields that start immutable should remain so as requirements grow.

x, y □ mutable (Step 5)

Similarly, x and y were initially final and became mutable when movement was introduced. The key principle: don't predict mutability needs; let TDD reveal them.

9. Testing Strategy

9.1 Principles

- One test per behaviour — each test verifies exactly one thing, making failures self-describing.
- Test names are complete sentences — e.g., shouldThrowExceptionWhenCommandsAreNull — readable as specifications.
- Arrange-Act-Assert structure followed consistently throughout all tests.
- No test depends on another test — all tests are fully isolated and can run in any order.
- Edge cases receive the same testing rigour as happy-path scenarios.

9.2 Test Coverage Summary

Test Category	Example Test Name	Step
Probe construction	shouldStoreInitialPosition	1
Direction initialisation	shouldStoreInitialDirection	2
Turn left	shouldTurnLeftFromNorthToWest	3
Turn right	shouldTurnRightFromNorthToEast	4
Move forward	shouldMoveForwardWhenFacingNorth	5
Move backward	shouldMoveBackwardWhenFacingNorth	6
Boundary — no move	shouldNotMoveOutsideGrid	7
Obstacle avoidance	shouldNotMoveIntoObstacle	8
Command execution	shouldExecuteCommandSequence	10
Null commands	shouldThrowExceptionWhenCommandsAreNull	11
Empty commands	shouldDoNothingWhenCommandsAreEmpty	12
Invalid command char	shouldThrowExceptionForInvalidCommand	10
Whitespace in commands	shouldIgnoreSpacesInCommands	14
Lowercase commands	shouldAcceptLowercaseCommands	19
Null direction	shouldThrowExceptionWhenDirectionIsNull	16b
Null grid	shouldThrowExceptionWhenGridIsNull	17
Start out of bounds	shouldThrowExceptionWhenStartingOutsideBounds	18
Start on obstacle	shouldThrowExceptionWhenStartingOnObstacle	20
Invalid grid width	shouldThrowExceptionForZeroWidth	21
Invalid grid height	shouldThrowExceptionForNegativeHeight	21

Obstacle outside grid	shouldThrowExceptionWhenObstacleOutsideGrid	22
Duplicate obstacle	shouldHandleDuplicateObstaclesGracefully	23

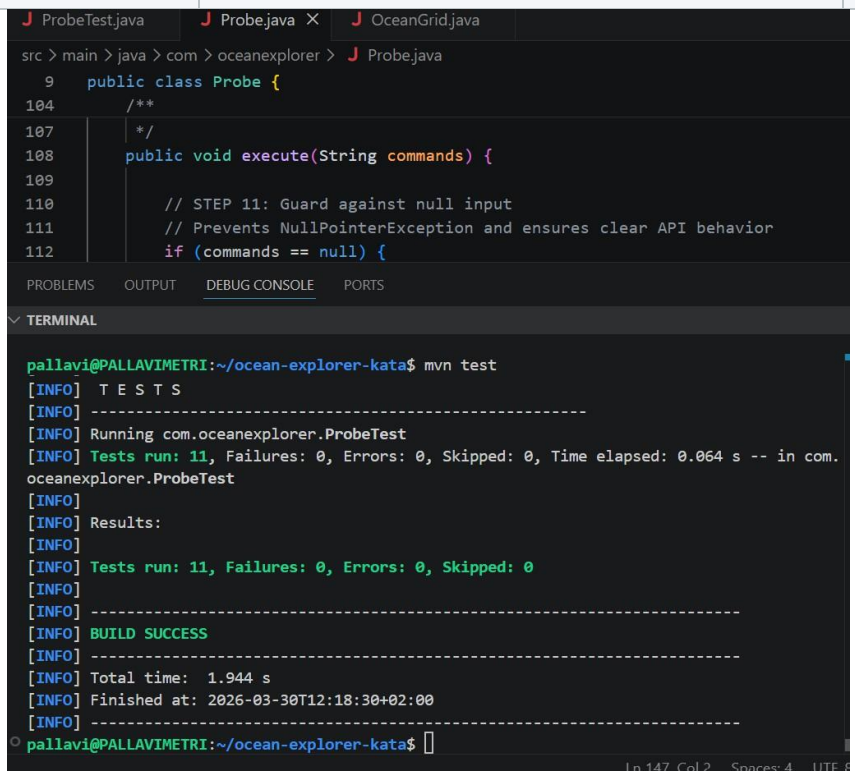


Figure 12 — Full test suite passing in ProbeTest.java

10. Non-Functional Considerations

Maintainability	Each class has one reason to change. Probe changes when navigation behaviour changes; OceanGrid changes when grid rules change; Coordinate changes when position representation changes. This isolation means modifications are localised and safe.
Readability	Method names are verb phrases (moveForward, turnLeft, hasObstacle). Test names are complete behavioural specifications. Switch expressions replace verbose if-else chains. The codebase reads like well-structured documentation.
Extensibility	The Direction enum can be extended with diagonal directions without changing movement logic. OceanGrid can be subclassed to introduce wrapping (torus-topology) grids. The execute() method can accept new command characters by adding cases to processCommand().
Scalability	Obstacle storage uses a HashSet for O(1) insertion and lookup regardless of the number of obstacles. The system scales to large grids without performance degradation from the obstacle-check path.
Testability	All classes are designed for dependency injection. OceanGrid is passed into Probe rather than created internally, enabling test doubles if needed in future. No static state exists.
Robustness	Every public method is defensive. No method trusts its caller to pass valid inputs. All invalid states are caught at the earliest possible point, typically at construction. The system follows defensive programming principles. Public methods validate inputs and handle invalid states early, typically during object construction, ensuring stability and predictable behavior

11. How AI Was Used

Artificial Intelligence tooling was used as a **guided assistant** during this project — not as an autonomous code generator. The distinction is important and reflects a principled engineering approach to AI-assisted development.

11.1 The Human-Led Model

Every design decision, every class structure, every test name, and every refactoring choice was made by the engineer. AI was consulted as a knowledgeable collaborator — similar to pair programming with a senior developer — to explain concepts, validate thinking, and suggest alternatives when stuck.

Concept Clarification: When TDD principles or OOP concepts required deeper explanation, AI was used to elaborate — for example, explaining why a Set is preferable to a List for obstacle storage.

Design Validation: Proposed designs were described to the AI to receive feedback on their adherence to SOLID principles and to identify potential issues before implementation.

Error Diagnosis: When build failures were unclear (e.g., unexpected test output), AI helped interpret error messages and identify root causes — particularly for incorrect test assertions.

Refactoring Guidance: AI helped identify duplication patterns (e.g., the shared move() helper) and suggested idiomatic Java approaches to resolving them.

Step Narration: AI assisted in structuring the development journey as a coherent narrative for this document, converting raw notes into polished engineering prose.

11.2 What AI Did Not Do

AI did not write production code autonomously. Every method was manually implemented, understood before development, and validated against a failing test first. The TDD discipline ensured human control at every step — no feature was added without a clearly defined and understood test.I did use AI to generate this report.

> *This represents the most effective model of AI collaboration in engineering: AI as an accelerator of understanding, not a replacement for thought.*

12. Conclusion

The Ocean Explorer Kata demonstrates the application of disciplined Test-Driven Development to produce a clean, well-structured, and thoroughly tested Java API. Over 25 incremental steps, the system evolved from a simple class storing two integers to a fully validated, multi-class architecture handling navigation, boundaries, obstacles, and command parsing.

Engineering Strengths Demonstrated

- TDD discipline — every feature justified by a failing test, no speculative code written.
- Clean class designs four cohesive classes with clearly defined non-overlapping responsibilities.
- Defensive programming — all inputs validated, all failure modes handled explicitly.
- Incremental complexity was introduced only when tests demanded it.
- Refactoring maturity — duplication recognized and eliminated without regression.
- Test quality — test names as specifications, one assertion per test, full edge-case coverage.
- Design evolution — architecture allowed to grow naturally rather than being over-designed upfront.

Reflection

The most valuable insight from this exercise is that **TDD is not a testing strategy, it is a design strategy**. Writing the test first forced clear thinking about each method's contract before any implementation decisions were made. The result is code that is not just correct, but *demonstrably correct* every behavior is pinned by a test that will catch regression immediately.

The codebase reflects engineering values that scale beyond katas: clear intent, predictable behavior, and resistance to unintended change. These are the qualities that distinguish reliable production software from code that merely works today.

```
pallavi@PALLAVIMETRI:~/ocean-explorer-kata$ mvn test
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 25, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.252 s
[INFO] Finished at: 2026-03-30T15:37:23+02:00
[INFO] ------------------------------------------------------------------------
pallavi@PALLAVIMETRI:~/ocean-explorer-kata$
```

Figure 13 — Final full test suite: all tests passing

Ocean Explorer Kata · NatWest Software Engineer Submission · Java 17 · JUnit 5 · Maven · TDD